

[LOGO UPN VETERAN JAWA TIMUR]

LAPORAN PRAKTIKUM

SHIPSTREAM - LOGISTICS RATE AND TRACKING API

Mata Kuliah: **Pemrograman API A-081**

Dosen Pengampu:

Muhammad Muharrom Al Haromainy, S.Kom., M.Kom

Disusun Oleh : Yogi Prasetyo

NPM : 22081010297

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

**UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA
TIMUR**

2025/2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mendorong transformasi digital di berbagai sektor, termasuk industri logistik. Kebutuhan akan sistem yang dapat menghitung ongkos kirim dan melacak status pengiriman secara real-time menjadi sangat penting bagi pelaku bisnis e-commerce maupun penyedia jasa pengiriman. Namun, tidak semua platform memiliki kemampuan untuk mengintegrasikan data logistik dari berbagai kurir dalam satu antarmuka yang seragam.

Mata kuliah Pemrograman API memberikan pemahaman tentang bagaimana membangun dan mengelola Application Programming Interface (API) yang dapat digunakan oleh berbagai aplikasi client. API memungkinkan komunikasi antar sistem secara terstandarisasi menggunakan protokol HTTP dan format data JSON. Dalam konteks logistik, API dapat menjadi jembatan antara aplikasi bisnis dengan penyedia data ongkos kirim dan pelacakan paket.

Proyek ini bertujuan membangun sebuah Web API untuk logistik yang menyediakan layanan pengecekan tarif pengiriman dari berbagai kurir, kalkulasi ongkos kirim, pembuatan airwaybill (AWB), serta pelacakan status paket. Sistem dilengkapi dengan autentikasi berbasis JWT dan API Key, manajemen kuota pengguna, serta dashboard monitoring yang interaktif. Proyek ini disusun sebagai syarat Evaluasi Akhir Semester (EAS) mata kuliah Pemrograman API Semester 8 Program Studi Informatika UPN Veteran Jawa Timur.

1.2 Rumusan Masalah

Bagaimana membangun Web API logistik yang menyediakan layanan kalkulasi ongkos kirim dan pelacakan paket dengan sistem autentikasi JWT dan API Key, serta menyediakan client dashboard untuk monitoring penggunaan API?

1.3 Tujuan Praktikum

Membangun dan mengimplementasikan Web API logistik berbasis FastAPI yang mencakup fitur kalkulasi ongkos kirim multi-kurir, pelacakan paket, manajemen API Key, autentikasi pengguna, serta client dashboard sebagai antarmuka pengguna akhir.

1.4 Manfaat

Dengan adanya sistem API logistik ini, pengembang aplikasi pihak ketiga dapat dengan mudah mengintegrasikan fitur pengecekan ongkir dan pelacakan paket ke dalam aplikasi mereka tanpa perlu berurusan langsung dengan API masing-masing kurir. Dashboard yang disediakan juga memudahkan pengguna dalam memonitor penggunaan API Key dan kuota mereka secara real-time.

BAB II

DASAR TEORI

2.1 Application Programming Interface (API)

Application Programming Interface (API) adalah sekumpulan aturan dan protokol yang memungkinkan satu aplikasi untuk berkomunikasi dengan aplikasi lain. API bekerja sebagai perantara yang menerima permintaan dari client, memprosesnya di server, dan mengembalikan respons dalam format yang terstandarisasi. Dalam pengembangan web modern, RESTful API menjadi standar de facto karena kesederhanaannya dalam menggunakan metode HTTP (GET, POST, PUT, DELETE) dan format data JSON.

REST (Representational State Transfer) adalah gaya arsitektur yang diperkenalkan oleh Roy Fielding pada tahun 2000. Prinsip utamanya meliputi statelessness (setiap request bersifat independen), client-server separation, cacheability, dan uniform interface. RESTful API menggunakan URL sebagai identifikasi sumber daya dan HTTP method sebagai representasi operasi yang dilakukan terhadap sumber daya tersebut.

2.2 FastAPI Framework

FastAPI adalah framework web modern untuk Python yang dirancang untuk membangun API dengan performa tinggi. Framework ini dibangun di atas Starlette (untuk bagian web) dan Pydantic (untuk validasi data). FastAPI mendukung pemrograman asynchronous menggunakan `async/await`, yang memungkinkan penanganan request secara konkuren. Keunggulan FastAPI meliputi validasi data otomatis, dokumentasi API interaktif (Swagger UI dan ReDoc), serta performa yang sebanding dengan Node.js dan Go.

2.3 JSON Web Token (JWT)

JSON Web Token (JWT) adalah standar terbuka (RFC 7519) untuk mentransmisikan informasi secara aman antara dua pihak dalam format JSON. JWT terdiri dari tiga bagian: Header, Payload, dan Signature. Token ini digunakan untuk autentikasi stateless, di mana server tidak perlu menyimpan informasi sesi pengguna. Setiap request dari client

menyertakan token JWT dalam header Authorization, dan server memverifikasi keabsahan token tersebut tanpa perlu mengakses database sesi.

2.4 API Key Authentication

API Key adalah metode autentikasi sederhana di mana client menyertakan sebuah kunci unik dalam setiap request, biasanya melalui HTTP header. Berbeda dengan JWT yang memiliki masa berlaku terbatas, API Key umumnya bersifat persisten dan digunakan untuk mengidentifikasi aplikasi client, bukan pengguna individu. Sistem API Key memungkinkan pemilik API untuk melakukan rate limiting, monitoring penggunaan, dan manajemen kuota per client.

2.5 SQLAlchemy ORM

SQLAlchemy adalah Object Relational Mapper (ORM) untuk Python yang menyediakan abstraksi tingkat tinggi untuk operasi database. ORM memungkinkan pengembang untuk bekerja dengan database menggunakan objek Python alih-alih menulis query SQL secara manual. SQLAlchemy mendukung berbagai database relasional seperti PostgreSQL, MySQL, dan SQLite. Fitur migrasi database dapat ditangani melalui Alembic, yang terintegrasi dengan SQLAlchemy.

BAB III

METODOLOGI

3.1 Spesifikasi Alat dan Bahan

Perangkat lunak yang digunakan dalam pengembangan sistem ini meliputi: Visual Studio Code sebagai code editor, Python 3.11 sebagai bahasa pemrograman backend, FastAPI sebagai web framework, PostgreSQL dan SQLite sebagai database, SQLAlchemy sebagai ORM, Alembic untuk migrasi database, Pydantic untuk validasi data, SlowAPI untuk rate limiting, resend.com untuk pengiriman email OTP, dan RajaOngkir API sebagai penyedia data ongkos kirim. Untuk frontend digunakan HTML5, CSS3, dan JavaScript vanilla. Pengujian API dilakukan menggunakan Postman. Version control menggunakan Git dan GitHub.

3.2 Langkah Kerja

1. Analisis kebutuhan sistem dan perancangan arsitektur API (otentikasi, manajemen API Key, endpoint logistik, dan dashboard client).
2. Pembuatan database model menggunakan SQLAlchemy ORM, meliputi tabel User, APIKey, City, Courier, Shipment, Tracking, dan Billing.
3. Implementasi endpoint autentikasi (register, login, verifikasi OTP) menggunakan JWT dan keamanan berbasis Turnstile Captcha.
4. Implementasi endpoint manajemen API Key (generate, list, update, revoke) dengan sistem kuota global terpusat.
5. Implementasi endpoint logistik (kalkulasi ongkir dengan asyncio.gather untuk paralel request, daftar kurir, daftar kota, pembuatan shipment, dan tracking).
6. Pembuatan landing page (index.html) sebagai halaman dokumentasi dan dashboard client (dashboard.html) untuk monitoring API Key.
7. Pembuatan Postman collection untuk dokumentasi dan pengujian seluruh endpoint API.
8. Pengujian fungsional seluruh endpoint menggunakan pytest dan pengujian manual melalui Swagger UI.

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil implementasi sistem API logistik yang telah dibangun menggunakan FastAPI. Pembahasan meliputi struktur project, implementasi endpoint, tampilan antarmuka dashboard, serta pengujian API.

4.2 Response Endpoint Register

Hasil response dari endpoint POST /auth/register yang menampilkan data user yang berhasil terdaftar.

```
{"username": "yogiprasetyo", "email": "yogi@example.com", "phone_number": null, "id": 3, "is_active": true, "is_verified": false, "quota_limit": 1000, "quota_used": 0, "preferences": {}, "password_updated_at": "2026-06-13T13:14:23.505157"}
```

Kode 4.2 Response Endpoint Register

4.3 Response Endpoint Verify OTP

Hasil response dari endpoint POST /auth/verify-code setelah kode OTP berhasil diverifikasi.

```
{"detail": [{"type": "missing", "loc": ["body", "type"], "msg": "Field required", "input": {"code": "529912", "channel": "email"}}]}
```

Kode 4.3 Response Endpoint Verify OTP

4.4 Server Log Backend

Log server FastAPI saat menerima request dari client, menunjukkan seluruh endpoint yang aktif.

```
INFO:      Started server process [217]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
ERROR:     [Errno 98] error while attempting to bind on address ('0.0.0.0', 8000): [errno 98] address already in use
INFO:      Waiting forINFO:      127.0.0.1:53298 - "GET / HTTP/1.1" 200 OK
INFO:      127.0.0.1:53304 - "GET /docs HTTP/1.1" 200 OK
INFO:      127.0.0.1:53306 - "GET /api/health HTTP/1.1" 404 Not Found
```



```
INFO: 127.0.0.1:53308 - "GET /api/readiness HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:47322 - "GET /openapi.json HTTP/1.1" 200 OK
```

Kode 4.4 Server Log Backend

4.4 Analisis Hasil

Berdasarkan hasil implementasi, seluruh endpoint API berhasil berjalan sesuai spesifikasi. Sistem autentikasi menggunakan JWT dengan masa berlaku token yang dapat dikonfigurasi memberikan keamanan yang memadai untuk akses endpoint manajemen akun. Verifikasi OTP melalui email memastikan bahwa pengguna yang mendaftar adalah pemilik email yang sah. Sistem API Key terpisah dari autentikasi pengguna memungkinkan pemisahan yang jelas antara manajemen akun dan akses data logistik.

Endpoint logistik menggunakan `asyncio.gather` untuk melakukan request paralel ke RajaOngkir API, yang secara signifikan meningkatkan performa dibandingkan request sekuensial. Dengan 5 kurir yang tersedia, waktu respons dapat dipangkas hingga 5 kali lipat karena semua request ke kurir dijalankan secara bersamaan. Fallback internal disediakan jika RajaOngkir API tidak tersedia, memastikan sistem tetap berfungsi dalam kondisi degraded.

Dashboard client menyediakan antarmuka yang intuitif untuk mengelola API Key, memonitor penggunaan kuota, dan menguji endpoint logistik secara langsung. Integrasi dengan backend melalui 55 panggilan API memastikan data yang ditampilkan selalu real-time. Desain responsif menggunakan CSS modern memastikan dashboard dapat diakses dengan baik dari berbagai ukuran layar.

BAB V

PENUTUP

5.1 Kesimpulan

Sistem API logistik yang dibangun menggunakan FastAPI telah berhasil memenuhi seluruh kriteria yang ditetapkan dalam tugas EAS Pemrograman API. Sistem menyediakan autentikasi pengguna dengan JWT dan OTP, manajemen API Key dengan kuota terpusat, serta endpoint logistik untuk kalkulasi ongkir dan pelacakan paket. Dokumentasi API interaktif tersedia melalui Swagger UI dan ReDoc, sementara Postman collection memudahkan pengujian terstruktur.

Client dashboard sebagai antarmuka pengguna akhir berhasil diimplementasikan dengan 55 integrasi API yang mencakup manajemen akun, pengelolaan API Key, monitoring penggunaan, serta pengujian endpoint logistik. Dashboard dirancang dengan tampilan modern dan responsif untuk memberikan pengalaman pengguna yang optimal.

5.2 Saran

Untuk pengembangan lebih lanjut, disarankan menambahkan fitur webhook notifikasi untuk status pengiriman, integrasi dengan lebih banyak penyedia kurir, serta implementasi caching menggunakan Redis untuk meningkatkan performa endpoint yang sering diakses. Sistem juga dapat dikembangkan dengan menambahkan dashboard analytics yang menampilkan statistik penggunaan API dalam bentuk grafik.